LAB 2 · LECTURE 2

Lecture 2 Lab: Embeddings and Visualization

Vector Databases Course | Estimated time: 45-60 minutes | Platform: Google Colab (free)

Goal

You will embed 24 short sentences from four topics, measure how similar they are, and plot them in two dimensions with PCA and t-SNE. You will then compare two embedding models and probe where embeddings fail.

- Generate sentence embeddings with sentence-transformers and inspect their shape and norm.
- Quantify whether same-topic sentences are closer than different-topic sentences.
- Visualize the space with PCA and t-SNE, and tune t-SNE's perplexity.
- Compare two models, test a multilingual pair, and find failure cases such as negation.
- Save embeddings together with the model name, as recommended in the lecture.

Setup

Create a new Colab notebook and run each block in its own cell, in order. No GPU is needed.

Cell 1: install and import

```
!pip install -q sentence-transformers scikit-learn matplotlib pandas numpy

import time, json
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sentence_transformers import SentenceTransformer
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
```

Part 1: The data

Six sentences per topic. Two pairs deliberately say the same thing in different words, as in Lecture 1.

Cell 2: sentences and topic labels

```
data = [
 ("travel", "Affordable hotels near the beach"),
 ("travel", "Cheap flights to Cairo"),
 ("travel", "Guided tours of the pyramids"),
 ("travel", "Budget hostel for backpackers"),
 ("travel", "Book a sightseeing trip to ancient monuments"),
 ("travel", "Best time of year to visit Luxor"),
 ("tech",   "How to reset your account password"),
 ("tech",   "I forgot my login credentials"),
 ("tech",   "Laptop battery drains too quickly"),
 ("tech",   "My computer's charge does not last long"),
 ("tech",   "The app crashes when I open settings"),
 ("tech",   "Wi-Fi keeps disconnecting every few minutes"),
 ("food",   "Best restaurants for seafood lovers"),
 ("food",   "Where to eat fresh fish by the sea"),
 ("food",   "A simple recipe for tomato soup"),
 ("food",   "How to bake sourdough bread at home"),
 ("food",   "Vegetarian dishes with chickpeas"),
 ("food",   "Street food worth trying in Alexandria"),
 ("health", "Tips for improving your sleep"),
 ("health", "How much water should I drink daily"),
 ("health", "Easy stretches for lower back pain"),
 ("health", "Ways to reduce stress at work"),
 ("health", "A beginner's guide to running"),
 ("health", "Foods that support heart health"),
]
topics = [t for t, _ in data]
texts  = [s for _, s in data]
print(len(texts), "sentences,", len(set(topics)), "topics")
```

Part 2: Generate embeddings

Cell 3: encode and inspect

```
model = SentenceTransformer("all-MiniLM-L6-v2")
vecs = model.encode(texts, normalize_embeddings=True)

print("shape        :", vecs.shape)              # (24, 384)
print("first 6 nums :", vecs[0][:6].round(3))
print("L2 norm      :", np.linalg.norm(vecs[0]))   # 1.0 after normalizing
print("max length   :", model.max_seq_length)
```

YOUR TURN Which number in the shape is the number of sentences and which is the dimension? Re-run the cell with `normalize_embeddings=False` and report the norm.

Part 3: Measure similarity

For unit vectors, the dot product equals cosine similarity (Lecture 3 explains why). One matrix product gives all pairwise scores.

Cell 4: nearest neighbors and topic separation

```
sims = vecs @ vecs.T

def neighbors(i, k=3):
    order = np.argsort(-sims[i])[1:k+1]           # skip itself
    return [(texts[j], round(float(sims[i, j]), 2), topics[j]) for j in order]

for i in [0, 6, 12]:
    print(texts[i])
    for row in neighbors(i):
        print("   ", row)

def separation(v):
    s = v @ v.T
    same = np.array(topics)[:, None] == np.array(topics)[None, :]
    off = ~np.eye(len(v), dtype=bool)
    return s[same & off].mean(), s[~same].mean()

intra, inter = separation(vecs)
print(f"same topic: {intra:.3f}   different topic: {inter:.3f}   gap: {intra - inter:.3f}")
```

YOUR TURN Are the nearest neighbors of each sentence in the same topic? Find one sentence whose neighbor surprises you and explain why.

Part 4: Visualize with PCA

Cell 5: plotting helper and PCA

```
def plot2d(xy, title):
    plt.figure(figsize=(7, 5))
    for t in sorted(set(topics)):
        idx = [i for i, x in enumerate(topics) if x == t]
        plt.scatter(xy[idx, 0], xy[idx, 1], s=70, label=t)
    plt.legend(); plt.title(title); plt.grid(alpha=0.3); plt.show()

pca = PCA(n_components=2)
xy_pca = pca.fit_transform(vecs)
print("variance kept:", pca.explained_variance_ratio_.round(3), pca.explained_variance_ratio_.sum().round(
plot2d(xy_pca, "PCA of sentence embeddings")
```

YOUR TURN How much of the total variance do the two components keep? What does that tell you about how much information the 2-D plot hides?

Part 5: Visualize with t-SNE

Cell 6: t-SNE with different perplexities

```
for perp in [3, 8, 15]:
    tsne = TSNE(n_components=2, perplexity=perp, init="pca", random_state=42)
    plot2d(tsne.fit_transform(vecs), f"t-SNE (perplexity={perp})")
```

Perplexity must be smaller than the number of points (24 here). Very small values fragment clusters; large values blur them.

YOUR TURN Change `random_state` to 7 and re-run perplexity 8. What changes and what stays? Which conclusions about the plot are safe, and which are not (cluster sizes, gaps between clusters)?

Part 6: Compare two models

Cell 7: a bigger model

```
rows = []
for name in ["all-MiniLM-L6-v2", "all-mpnet-base-v2"]:
    m = SentenceTransformer(name)
    t0 = time.perf_counter()
    v = m.encode(texts, normalize_embeddings=True)
    dt = time.perf_counter() - t0
    intra, inter = separation(v)
    rows.append(dict(model=name, dims=v.shape[1], seconds=round(dt, 2),
                     same=round(intra, 3), different=round(inter, 3),
                     gap=round(intra - inter, 3)))
pd.DataFrame(rows)
```

YOUR TURN Complete the table below from your output, then state which model you would use for a small English FAQ search and why.

Model	Dims	Seconds	Same topic	Different	Gap
all-MiniLM-L6-v2					
all-mpnet-base-v2					

Part 7: Break it

Cell 8: negation, identifiers, and another language

```
def cos(a, b):
    va, vb = model.encode([a, b], normalize_embeddings=True)
    return round(float(va @ vb), 3)

print("negation  :", cos("The refund is allowed", "The refund is not allowed"))
print("unrelated :", cos("The refund is allowed", "Guided tours of the pyramids"))
print("ids       :", cos("Order A-4471 has shipped", "Order A-4472 has shipped"))
print("french    :", cos("I forgot my password", "J'ai oublie mon mot de passe"))
```

The French pair uses an English-only model, so it should score low. Now repeat it with a multilingual model:

```
mm = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
a, b = mm.encode(["I forgot my password", "J'ai oublie mon mot de passe"], normalize_embeddings=True)
print("multilingual french:", round(float(a @ b), 3))
```

YOUR TURN Write what each test shows. Which failure would hurt most in a search over legal contracts? In a search over product codes?

Part 8: Save embeddings the right way

Cell 9: store vectors with their model name

```
np.save("embeddings.npy", vecs)
meta = {"model": "all-MiniLM-L6-v2", "dims": int(vecs.shape[1]),
        "normalized": True, "texts": texts, "topics": topics}
json.dump(meta, open("embeddings_meta.json", "w"), indent=2)
print("saved", vecs.shape, "with metadata")
```

This habit prevents the most common production bug: querying an index with a different model than the one used to build it.

Reflection questions

- Why are individual embedding dimensions not interpretable, while distances are meaningful?
- t-SNE showed four clean clusters. Can you conclude that the topics are equally different from each other? Why or why not?
- A bigger model gave a larger gap on 24 sentences. Why is that not enough to choose it for production?
- Your search must handle names, SKUs and numbers well. What would you add to embeddings? (Lecture 9 returns to this.)

Deliverable

Submit the executed notebook with both plots, the completed table, the answers to each YOUR TURN box, and short answers to the reflection questions.

Troubleshooting

- **Model download is slow:** it happens only on the first run of each model.
- **t-SNE ValueError about perplexity:** perplexity must be smaller than the number of samples.
- **Plots look different from classmates':** t-SNE depends on random_state; PCA is deterministic.
- **Scores differ slightly from the handout:** library versions vary; the ranking should look similar.